

Log in to a Remote System

In Linux, the most common way to get a shell prompt on a remote system is to use Secure Shell (SSH). Most Linux systems (including Red Hat Enterprise Linux) and macOS provide the `openssh` command-line program `ssh` for this purpose.

In this example, a user with a shell prompt on the host machine uses `ssh` to log in to the remote Linux system `remotehost` as the user `remoteuser`:

```
[user@host ~]$ ssh remoteuser@remotehost
remoteuser@remotehost's password: password
[remoteuser@remotehost ~]$
```

The `ssh` command encrypts the connection to secure the communication against eavesdropping or hijacking of the passwords and content.

In the next example, a user with a shell prompt on the host machine logs in to `remotehost` as `remoteuser` with `ssh`, by using the public key authentication method. The `ssh` command `-i` option is used to specify the user's private key file, which is `mylab.pem`. The matching public key is already set up as an authorized key in the `remoteuser` account.

```
[user@host ~]$ ssh -i mylab.pem remoteuser@remotehost
[remoteuser@remotehost ~]$
```

Note

When you first log in to a new machine, you are prompted with a warning from `ssh` that it cannot establish the authenticity of the host:

```
[user@host ~]$ ssh -i mylab.pem remoteuser@remotehost
The authenticity of host 'remotehost (192.0.2.42)' can't be established.
ECDSA key fingerprint is 47:bf:82:cd:fa:68:06:ee:d8:83:03:1a:bb:29:14:a3.
Are you sure you want to continue connecting (yes/no)? yes
[remoteuser@remotehost ~]$
```

Log Out from a Remote System

When you are finished with the shell and want to quit, you can choose one of several ways to end the session. You can enter the `exit` command to terminate the current shell session. Alternatively, finish a session by pressing **Ctrl+D**.

The following example shows a user who logs out of an SSH session:

```
[remoteuser@remotehost ~]$ exit
logout
Connection to remotehost closed.
[user@host ~]$
```

Guided Exercise: Access the Command Line with the Desktop

In this exercise, you log in through the graphical display manager as a regular user to become familiar with the GNOME Standard desktop environment that GNOME 40 provides.

Outcomes

- Log in to a Linux system by using the GNOME 40 desktop environment.
- Run commands from a shell prompt in a terminal program.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command prepares your environment and ensures that all required resources are available.

```
[student@workstation ~]$ lab start cli-desktop
```

Instructions

1. Log in to workstation as student with student as the password.
 1. On workstation, at the GNOME login screen, click the student user account. Enter student when prompted for the password.
 2. Press **Enter**.
2. Change the password for student from student to 55TurnK3y.

Important

The finish script resets the password for the student user to student. You must execute the script at the end of the exercise.

1. Open a Terminal window and use the `passwd` command at the shell prompt.

In the virtual learning environment with a visual keyboard, press the **Super** key twice to enter the **Activities** overview.

Type `terminal` and then press **Enter** to start Terminal.

2. In the Terminal window that opens, type `passwd` at the shell prompt. Change the student password from `student` to `55TurnK3y`.

```
3. [student@workstation ~]$ passwd
4. Changing password for user student.
5. Current password: student
6. New password: 55TurnK3y
7. Retype new password: 55TurnK3y
```

```
passwd: all authentication tokens updated successfully.
```

3. Log out and log back in as student with `55TurnK3y` as the password to verify the changed password.
 1. Click the system menu in the upper-right corner.
 2. Select **Power Off/Log Out** → **Log Out**.
 3. Click **Log Out** in the confirmation dialog box that is displayed.
 4. At the GNOME login screen, click the student user account.
Enter `55TurnK3y` when prompted for the password.
 5. Press **Enter**.
4. Lock the screen.
 1. From the system menu in the upper-right corner, press the **Lock** button.
5. Unlock the screen.
 1. Press **Enter** to unlock the screen.
 2. In the **Password** field, enter `55TurnK3y` as the password.
 3. Press **Enter**.
6. Determine how to shut down workstation from the graphical interface, but **Cancel** the operation without shutting down the system.

1. From the system menu in the upper-right corner, select **Power Off/Log Out** → **Power Off**. A dialog box is displayed with the options to either **Cancel** or **Power Off** the machine.
2. Click **Cancel** in the dialog box that is displayed.

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-desktop
```

This concludes the section.

Execute Commands with the Bash Shell

Objectives

Save time when running commands from a shell prompt with Bash shortcuts.

Basic Command Syntax

The GNU Bourne-Again Shell (bash) is a program that interprets commands that the user types. Each string that is typed into the shell can have up to three parts: the command, options (which usually begin with a hyphen - or double hyphen - - characters), and arguments. Each word that is typed into the shell is separated from other words with spaces. Commands are the names of programs that are installed on the system. Each command has its options and arguments.

When you are ready to execute a command, press the **Enter** key. Type each command on a separate line. The command output is displayed before the following shell prompt appears.

```
[user@host ~]$ whoami
user
[user@host ~]$
```

To type more than one command on a single line, use the semicolon (;) as a command separator. A semicolon is a member of a class of characters called *metacharacters* that have a special interpretation for `bash`. In this case, the output of both commands is displayed before the following shell prompt appears.

The following example shows how to combine two commands (`command1` and `command2`) on the command line.

```
[user@host ~]$ command1 ; command2
command1 output
command2 output
[user@host ~]$
```

Write Simple Commands

The `date` command displays the current date and time. The superuser or a privileged user can also use the `date` command to set the system clock. Use the plus sign (+) as an argument to specify a format string for the `date` command.

```
[user@host ~]$ date
Sun Feb 27 08:32:42 PM EST 2022
[user@host ~]$ date +%R
20:33
[user@host ~]$ date +%x
02/27/2022
```

The `passwd` command with no options changes the current user's password. To change the password, first specify the original password for the account. By default, the `passwd` command is configured to require a strong password, to consist of lowercase letters, uppercase letters, numbers, and symbols, and not to be based on a dictionary word. A superuser or privileged user can use the `passwd` command to change another user's password.

```
[user@host ~]$ passwd
Changing password for user user.
Current password: old_password
New password: new_password
```

```
Retype new password: new_password  
passwd: all authentication tokens updated successfully.
```

Linux does not require file name extensions to classify files by type. The `file` command scans the compiled header of a file for a 2-digit magic number and displays its type. Text files are recognized because they are not compiled.

```
[user@host ~]$ file /etc/passwd  
/etc/passwd: ASCII text  
[user@host ~]$ file /bin/passwd  
/bin/passwd: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=a467cb9c8fa7306d41b96a820b0178f3a9c66055, for GNU/Linux 3.2.0, stripped  
[user@host ~]$ file /home  
/home: directory
```

View the Contents of Files

The `cat` command is often used in Linux. Use this command to create single or multiple files, view the contents of files, concatenate the contents from various files, and redirect contents of the file to a terminal or to files.

The following example shows how to view the contents of the `/etc/passwd` file:

```
[user@host ~]$ cat /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
bin:x:1:1:bin:/bin:/sbin/nologin  
daemon:x:2:2:daemon:/sbin:/sbin/nologin  
adm:x:3:4:adm:/var/adm:/sbin/nologin  
...output omitted...
```

To display the contents of multiple files, add the file names to the `cat` command as arguments:

```
[user@host ~]$ cat file1 file2  
Hello World!!  
Introduction to Linux commands.
```

Some files are long and might need more space to be displayed than the terminal provides. The `cat` command does not display the contents of a file as pages. The `less` command displays one page of a file at a time and you can scroll at your leisure.

Use the `less` command to page forward and backward through longer files than can fit on one terminal window. Use the **UpArrow** key and the **DownArrow** key to scroll up and down. Press **q** to exit the command.

The `head` and `tail` commands display the beginning and the end of a file, respectively. By default, these commands display 10 lines of the file, but they both have a `-n` option to specify a different number of lines.

```
[user@host ~]$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
[user@host ~]$ tail -n 3 /etc/passwd
gdm:x:42:42:./var/lib/gdm:/sbin/nologin
gnome-initial-setup:x:980:978:./run/gnome-initial-setup:/sbin/nologin
dnsmasq:x:979:977:Dnsmasq DHCP and DNS server:/var/lib/dnsmasq:/sbin/nologin
```

The `wc` command counts lines, words, and characters in a file. Use the `-l`, `-w`, or `-c` options to display only the given number of lines, words, or characters, respectively.

```
[user@host ~]$ wc /etc/passwd
41   98 2338 /etc/passwd
[user@host ~]$ wc -l /etc/passwd ; wc -l /etc/group
```

```
41 /etc/passwd
63 /etc/group
[user@host ~]$ wc -c /etc/group /etc/hosts
883 /etc/group
114 /etc/hosts
997 total
```

Understand Tab Completion

With tab completion, users can quickly complete commands or file names after typing enough at the prompt to make it unique. If the typed characters are not unique, then pressing the **Tab** key twice displays all commands that begin with the typed characters.

```
[user@host ~]$ pasTab+Tab
passwd      paste      pasuspender

[user@host ~]$ passTab
[user@host ~]$ passwd
Changing password for user user.
Current password:
```

Press **Tab** twice.

Press **Tab** once.

Tab completion helps to complete file names when typing them as arguments to commands. Press **Tab** to complete as much of the file name as possible.

Pressing **Tab** a second time causes the shell to list all files that the current pattern matches. Type additional characters until the name is unique, and then use tab completion to complete the command.

```
[user@host ~]$ ls /etc/pasTab

[user@host ~]$ ls /etc/passwdTab
passwd  passwd-
```


Press **Tab** once.

Press **Tab** once.

Use the `useradd` command to create users on the system. The `useradd` command has many options that might be hard to remember. By using tab completion, you can complete the option name with minimal typing.

```
[root@host ~]# useradd --Tab+Tab
--badnames          --gid              --no-log-init      --shell
--base-dir          --groups          --non-unique       --skel
--btrfs-subvolume-home --help            --no-user-group    --system
--comment           --home-dir        --password         --uid
--create-home       --inactive        --prefix           --user-group
--defaults          --key             --root
--expiredate        --no-create-home  --selinux-user
```

Press **Tab** twice.

Write a Long Command on Multiple Lines

Commands with many options and arguments can quickly become long and are automatically wrapped by the command window when the cursor reaches the right margin. Instead, type a long command by using more than one line for easier reading.

To write one command in more than one line, use a backslash character (`\`), which is referred to as the *escape character*. The backslash character ignores the meaning of the following character.

Previously, you learned that to complete a command entry, you press the **Enter** key, the newline character. By escaping the newline character, the shell moves to a new command line without executing the command. This way, the shell acknowledges the request by displaying a continuation prompt on an empty new line, which is known as the secondary prompt, and uses the greater-than character (`>`) by default. Commands can continue over many lines.

One issue with the secondary prompt's use of the greater-than character (`>`) is that new learners might mistakenly insert it as part of the typed command. Then, the

shell interprets a typed greater-than character as *output redirection*, which the user did not intend. Output redirection is discussed in an upcoming chapter. This course book does not show secondary prompts in screen outputs, to avoid confusion. A user still sees the secondary prompt in their shell window, but the course material intentionally displays only the characters to be typed, as demonstrated in the following example.

```
[user@host ~]$ head -n 3 \  
/usr/share/dict/words \  
/usr/share/dict/linux.words  
==> /usr/share/dict/words <==  
1080  
10-point  
10th  
  
==> /usr/share/dict/linux.words <==  
1080  
10-point  
10th
```

Display the Command History

The `history` command displays a list of previously executed commands that are prefixed with a command number.

The exclamation point character (!) is a metacharacter to expand previous commands without retyping them. The `!number` command expands to the command that matches the specified number. The `!string` command expands to the most recent command that begins with the specified string.

```
[user@host ~]$ history  
...output omitted...  
23  clear  
24  who  
25  pwd  
26  ls /etc
```

```

27  uptime
28  ls -l
29  date
30  history
[user@host ~]$ !ls
ls -l
total 0
drwxr-xr-x. 2 student student 6 Feb 27 19:24 Desktop
...output omitted...
[user@host ~]$ !26
ls /etc
abrt                hosts                pulse
adjtime             hosts.allow          purple
aliases             hosts.deny            qemu-ga
...output omitted...

```

The arrow keys help to navigate through previous commands in the shell's history. The **UpArrow** edits the previous command in the history list. The **DownArrow** edits the next command in the history list. The **LeftArrow** and **RightArrow** move the cursor left and right in the current command from the history list so that you can edit the command before running it.

Use either the **Esc+.** or **Alt+.** key combination simultaneously to insert the last word of the previous command at the cursor's current location. The repeated use of the key combination replaces that text with the last word of earlier commands in history. The **Alt+.** key combination is particularly convenient, because you can hold down **Alt** and press **.** repeatedly to quickly cycle earlier commands.

Edit the Command Line

When used interactively, `bash` has a command-line editing feature. Use the text editor commands to move around and modify the currently typed command. Using the arrow keys to move within the current command and to step through the command history was introduced earlier in this section. The following table shows further powerful editing commands.

Table 2.1. Useful Command-line Editing Shortcuts

Shortcut	Description
Ctrl+A	Jump to the beginning of the command line.
Ctrl+E	Jump to the end of the command line.
Ctrl+U	Clear from the cursor to the beginning of the command line.
Ctrl+K	Clear from the cursor to the end of the command line.
Ctrl+LeftArrow	Jump to the beginning of the previous word on the command line.
Ctrl+RightArrow	Jump to the end of the next word on the command line.
Ctrl+R	Search the history list of commands for a pattern.

These command-line editing commands are the most helpful for new users. For other commands, refer to the `bash(1)` man page.

Lab: Access the Command Line

In this lab, you use the Bash shell to execute commands.

Outcomes

- Successfully run simple programs from the Bash shell command line.
- Execute commands to identify file types and to display parts of text files.
- Practice using some Bash command history shortcuts to more efficiently repeat commands or parts of commands.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command prepares your environment and ensures that all required resources are available.

```
[student@workstation ~]$ lab start cli-review
```

Instructions

1. Use the `date` command to display the current time and date.

```
2. [student@workstation ~]$ date
```

```
Mon Feb 28 01:57:25 PM PDT 2022
```

Hide Solution

3. Display the current time in 24-hour clock time (for example, 13:57). Hint: The format string that displays that output is %R.

1. Use the +%R argument with the date command to display the current time in 24-hour clock time.

```
2. [student@workstation ~]$ date +%R
```

```
13:58
```

4. Hide Solution

5. What kind of file is /home/student/zcat? Is it readable by humans?

1. Use the file command to determine its file type.

```
2. [student@workstation ~]$ file zcat
```

```
zcat: a /usr/bin/sh script, ASCII text executable
```

6. Hide Solution

7. Use the wc command and Bash shortcuts to display the size of the zcat file.

1. You can use the wc command to display the number of lines, words, and bytes in the zcat script. Instead of retyping the file name, use the Bash history shortcut **Esc+.** (the keys **Esc** and **.** pressed at the same time) to reuse the argument from the previous command.

```
2. [student@workstation ~]$ wc Esc+.
```

```
3. [student@workstation ~]$ wc zcat
```

```
51  299 1988 zcat
```

8. Hide Solution

9. Display the first 10 lines of the zcat file.

1. The head command displays the beginning of the file. Try the **Esc+.** shortcut again.

```
2. [student@workstation ~]$ head Esc+.
```

```
3. [student@workstation ~]$ head zcat
```

```
4. #!/bin/sh
```

```
5. # Uncompress files to standard output.
```

```
6.
```

```
7. # Copyright (C) 2007, 2010-2018 Free Software Foundation, Inc.
8.
9. # This program is free software; you can redistribute it and/or modify
10.# it under the terms of the GNU General Public License as published by
11.# the Free Software Foundation; either version 3 of the License, or
    # (at your option) any later version.
```

10.Hide Solution

11.Display the last 10 lines of the zcat file.

1. Use the `tail` command to display the last 10 lines of the `zcat` file.

```
2. [student@workstation ~]$ tail Esc+.
3. [student@workstation ~]$ tail zcat
4. With no FILE, or when FILE is -, read standard input.
5.
6. Report bugs to "<bug-gzip@gnu.org>."
7.
8. case $1 in
9.  --help)    printf '%s\n' "$usage"    || exit 1;;
10.--version) printf '%s\n' "$version" || exit 1;;
11.esac
12.
```

```
exec gzip -cd "$@"
```

12.Hide Solution

13.Repeat the previous command exactly with four or fewer keystrokes.

1. Repeat the previous command exactly with four or fewer keystrokes. Press the **UpArrow** key once to scroll back one command through the command history, and then press **Enter** (uses two keystrokes). An alternative would be to enter the shortcut command `!!` and then press **Enter** (uses four keystrokes) to run the most recent command in the command history. Try both methods.

```
2. [student@workstation]$ !!
3. tail zcat
```

```

4. With no FILE, or when FILE is -, read standard input.
5.
6. Report bugs to "<bug-gzip@gnu.org>."
7.
8. case $1 in
9. --help)    printf '%s\n' "$usage"    || exit 1;;
10. --version) printf '%s\n' "$version" || exit 1;;
11. esac
12.

```

```
exec gzip -cd "$@"
```

14.Hide Solution

15.Use the `tail` command `-n 20` option to display the last 20 lines in the file. Use command-line editing to accomplish this task with a minimal number of keystrokes.

1. Use **UpArrow** key to display the previous command. Next, use the **Ctrl+A** key combination to move the cursor to the beginning of the line. Next, use the **Ctrl+RightArrow** key combination to jump to the next word. Next, add the `-n 20` option and press **Enter** to execute the command.

```

2. [student@workstation ~]$ tail -n 20 zcat
3.  -l, --list          list compressed file contents
4.  -q, --quiet        suppress all warnings
5.  -r, --recursive    operate recursively on directories
6.  -S, --suffix=SUF   use suffix SUF on compressed files
7.      --synchronous  synchronous output (safer if system crashes, but slo
wer)
8.  -t, --test         test compressed file integrity
9.  -v, --verbose      verbose mode
10.      --help        display this help and exit
11.      --version     display version information and exit
12.
13. With no FILE, or when FILE is -, read standard input.
14.

```

```
15. Report bugs to "<bug-gzip@gnu.org>."
16.
17. case $1 in
18. --help)    printf '%s\n' "$usage"    || exit 1; exit;;
19. --version) printf '%s\n' "$version" || exit 1; exit;;
20. esac
21.
```

```
exec gzip -cd "$@"
```

16. Hide Solution

17. Use the shell history to run the `date +%R` command again.

1. Use the `history` command to display the list of previous commands and to identify the specific `date` command to execute. Use `!number` to run the command, where *number* is the command number to use from the output of the `history` command.

Your shell history might differ from the following example. Determine the command number to use based on the output of your own `history` command.

```
[student@workstation ~]$ history
1  date
2  date +%R
3  file zcat
4  wc zcat
5  head zcat
6  tail zcat
7  tail -n 20 zcat
8  history
[student@workstation ~]$ !2
date +%R
14:02
```

18. Hide Solution

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade cli-review
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish cli-review
```

This concludes the section.

Guided Exercise: Manage Files with Command-line Tools

In this exercise, you create, organize, copy, and remove files and directories.

Outcomes

- Create, organize, copy, and remove files and directories.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command prepares your environment and ensures that all required resources are available.

```
[student@workstation ~]$ lab start files-manage
```

Instructions

1. Log in to the servera machine as the student user. In the student user's home directory, create three subdirectories: `Music`, `Pictures`, and `Videos`.
 1. Use the `ssh` command to log in to the servera machine as the student user. The systems are configured to use SSH keys for authentication; therefore, a password is not required.

```
2. [student@workstation ~]$ ssh student@servera
```

3. ...output omitted...

```
[student@servera ~]$
```

4. In the student user's home directory, use the `mkdir` command to create three subdirectories: Music, Pictures, and Videos.

```
[student@servera ~]$ mkdir Music Pictures Videos
```

2. Use the `touch` command to create sets of empty practice files to use during this lab. In each set, replace x with the numbers 1 through 6.
- Create six files with names of the form `songx.mp3`.
 - Create six files with names of the form `snapx.jpg`.
 - Create six files with names of the form `filmx.avi`.

```
3. [student@servera ~]$ touch song1.mp3 song2.mp3 song3.mp3 \
4. song4.mp3 song5.mp3 song6.mp3
5. [student@servera ~]$ touch snap1.jpg snap2.jpg snap3.jpg \
6. snap4.jpg snap5.jpg snap6.jpg
7. [student@servera ~]$ touch film1.avi film2.avi film3.avi \
8. film4.avi film5.avi film6.avi
9. [student@servera ~]$ ls -l
10. total 0
11. -rw-r--r--. 1 student student 0 Mar  7 20:58 film1.avi
12. -rw-r--r--. 1 student student 0 Mar  7 20:58 film2.avi
13. -rw-r--r--. 1 student student 0 Mar  7 20:58 film3.avi
14. -rw-r--r--. 1 student student 0 Mar  7 20:58 film4.avi
15. -rw-r--r--. 1 student student 0 Mar  7 20:58 film5.avi
16. -rw-r--r--. 1 student student 0 Mar  7 20:58 film6.avi
17. drwxr-xr-x. 2 student student 6 Mar  7 20:58 Music
18. drwxr-xr-x. 2 student student 6 Mar  7 20:58 Pictures
19. -rw-r--r--. 1 student student 0 Mar  7 20:58 snap1.jpg
20. -rw-r--r--. 1 student student 0 Mar  7 20:58 snap2.jpg
21. -rw-r--r--. 1 student student 0 Mar  7 20:58 snap3.jpg
22. -rw-r--r--. 1 student student 0 Mar  7 20:58 snap4.jpg
23. -rw-r--r--. 1 student student 0 Mar  7 20:58 snap5.jpg
```

```
24. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap6.jpg
25. -rw-r--r--. 1 student student 0 Mar 7 20:58 song1.mp3
26. -rw-r--r--. 1 student student 0 Mar 7 20:58 song2.mp3
27. -rw-r--r--. 1 student student 0 Mar 7 20:58 song3.mp3
28. -rw-r--r--. 1 student student 0 Mar 7 20:58 song4.mp3
29. -rw-r--r--. 1 student student 0 Mar 7 20:58 song5.mp3
30. -rw-r--r--. 1 student student 0 Mar 7 20:58 song6.mp3
```

```
drwxr-xr-x. 2 student student 6 Mar 7 20:58 Videos
```

31. Move the song files (.mp3 extension) to the Music directory, the snapshot files (.jpg extension) to the Pictures directory, and the movie files (.avi extension) to the Videos directory.

```
32. [student@servera ~]$ mv song1.mp3 song2.mp3 song3.mp3 \
33. song4.mp3 song5.mp3 song6.mp3 Music
34. [student@servera ~]$ mv snap1.jpg snap2.jpg snap3.jpg \
35. snap4.jpg snap5.jpg snap6.jpg Pictures
36. [student@servera ~]$ mv film1.avi film2.avi film3.avi \
37. film4.avi film5.avi film6.avi Videos
38. [student@servera ~]$ ls -l Music Pictures Videos
39. Music:
40. total 0
41. -rw-r--r--. 1 student student 0 Mar 7 20:58 song1.mp3
42. -rw-r--r--. 1 student student 0 Mar 7 20:58 song2.mp3
43. -rw-r--r--. 1 student student 0 Mar 7 20:58 song3.mp3
44. -rw-r--r--. 1 student student 0 Mar 7 20:58 song4.mp3
45. -rw-r--r--. 1 student student 0 Mar 7 20:58 song5.mp3
46. -rw-r--r--. 1 student student 0 Mar 7 20:58 song6.mp3
47.
48. Pictures:
49. total 0
50. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap1.jpg
51. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap2.jpg
52. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap3.jpg
```

```
53. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap4.jpg
54. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap5.jpg
55. -rw-r--r--. 1 student student 0 Mar 7 20:58 snap6.jpg
56.
57. Videos:
58. total 0
59. -rw-r--r--. 1 student student 0 Mar 7 20:58 film1.avi
60. -rw-r--r--. 1 student student 0 Mar 7 20:58 film2.avi
61. -rw-r--r--. 1 student student 0 Mar 7 20:58 film3.avi
62. -rw-r--r--. 1 student student 0 Mar 7 20:58 film4.avi
63. -rw-r--r--. 1 student student 0 Mar 7 20:58 film5.avi
```

```
-rw-r--r--. 1 student student 0 Mar 7 20:58 film6.avi
```

64. Create three subdirectories for organizing your files, and name the subdirectories `friends`, `family`, and `work`. Use a single command to create all three subdirectories at the same time.

```
65. [student@servera ~]$ mkdir friends family work
66. [student@servera ~]$ ls -l
67. total 0
68. drwxr-xr-x. 2 student student 6 Mar 7 21:01 family
69. drwxr-xr-x. 2 student student 6 Mar 7 21:01 friends
70. drwxr-xr-x. 2 student student 108 Mar 7 21:00 Music
71. drwxr-xr-x. 2 student student 108 Mar 7 21:00 Pictures
72. drwxr-xr-x. 2 student student 108 Mar 7 21:00 Videos
```

```
drwxr-xr-x. 2 student student 6 Mar 7 21:01 work
```

73. Copy files that contain numbers 1 and 2 to the `friends` directory, and files that contain numbers 3 and 4 to the `family` directory. Keep in mind that you are making copies; therefore, the original files must remain in their original locations after you complete the step.

When you copy files from multiple locations into a single location, Red Hat recommends that you change to the destination directory before you copy

the files. Use the simplest path syntax, whether absolute or relative, to reach the source for each file management task.

1. Copy files that contain numbers 1 and 2 to the friends directory.

```
2. [student@servera ~]$ cd friends
3. [student@servera friends]$ cp ~/Music/song1.mp3 ~/Music/song2.mp3 \
4. ~/Pictures/snap1.jpg ~/Pictures/snap2.jpg ~/Videos/film1.avi \
5. ~/Videos/film2.avi .
6. [student@servera friends]$ ls -l
7. total 0
8. -rw-r--r--. 1 student student 0 Mar  7 21:02 film1.avi
9. -rw-r--r--. 1 student student 0 Mar  7 21:02 film2.avi
10. -rw-r--r--. 1 student student 0 Mar  7 21:02 snap1.jpg
11. -rw-r--r--. 1 student student 0 Mar  7 21:02 snap2.jpg
12. -rw-r--r--. 1 student student 0 Mar  7 21:02 song1.mp3
```

```
-rw-r--r--. 1 student student 0 Mar  7 21:02 song2.mp3
```

13. Copy files that contain numbers 3 and 4 to the family directory.

```
14. [student@servera friends]$ cd ../family
15. [student@servera family]$ cp ~/Music/song3.mp3 ~/Music/song4.mp3 \
16. ~/Pictures/snap3.jpg ~/Pictures/snap4.jpg ~/Videos/film3.avi \
17. ~/Videos/film4.avi .
18. [student@servera family]$ ls -l
19. total 0
20. total 0
21. -rw-r--r--. 1 student student 0 Mar  7 21:04 film3.avi
22. -rw-r--r--. 1 student student 0 Mar  7 21:04 film4.avi
23. -rw-r--r--. 1 student student 0 Mar  7 21:04 snap3.jpg
24. -rw-r--r--. 1 student student 0 Mar  7 21:04 snap4.jpg
25. -rw-r--r--. 1 student student 0 Mar  7 21:04 song3.mp3
```

```
-rw-r--r--. 1 student student 0 Mar  7 21:04 song4.mp3
```

74. Copy the family and friends directories and their contents to the work directory.

```
75. [student@servera family]$ cd ../work
76. [student@servera work]$ cp -r ~/family ~/friends .
77. [student@servera work]$ ls -l
78. total 0
79. drwxr-xr-x. 2 student student 108 Mar  7 21:05 family
```

```
drwxr-xr-x. 2 student student 108 Mar  7 21:05 friends
```

80. Your project tasks are now complete, and it is time to clean up the directories. Use the `rm -r` command to recursively delete the family, friends, and work directories and their contents.

```
81. [student@servera work]$ cd ..
82. [student@servera ~]$ rm -r family friends work
83. [student@servera ~]$ ls -l
84. total 0
85. drwxr-xr-x. 2 student student 108 Mar  7 21:00 Music
86. drwxr-xr-x. 2 student student 108 Mar  7 21:00 Pictures
```

```
drwxr-xr-x. 2 student student 108 Mar  7 21:00 Videos
```

87. Return to the workstation system as the student user.

```
88. [student@servera ~]$ exit
89. logout
90. Connection to servera closed.
```

```
[student@workstation ~]$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish files-manage
```

This concludes the section.

Make Links Between Files

Objectives

Make multiple file names reference the same file with hard links and symbolic (or "soft") links.

Manage Links Between Files

You can create multiple file names that point to the same file. These file names are called *links*.

You can create two types of links: a *hard link*, or a *symbolic link* (sometimes called a *soft link*). Each way has its advantages and disadvantages.

Create Hard Links

Every file starts with a single hard link, from its initial name to the data on the file system. When you create a hard link to a file, you create another name that points to that same data. The new hard link acts exactly like the original file name. After the link is created, you cannot tell the difference between the new hard link and the original name of the file.

You can determine whether a file has multiple hard links by using the `ls -l` command. One item that it reports is each file's *link count*, the number of hard links that the file has. In the next example, the link count of the `newfile.txt` file is 1. It has exactly one absolute path, which is the `/home/user/newfile.txt` location.

```
[user@host ~]$ pwd
/home/user
[user@host ~]$ ls -l newfile.txt
-rw-r--r--. 1 user user 0 Mar 11 19:19 newfile.txt
```

You can use the `ln` command to create a hard link (another file name) that points to an existing file. The command needs at least two arguments: a path to the existing file, and the path to the hard link that you want to create.

The following example creates a hard link called `newfile-hlink2.txt` for the existing `newfile.txt` file in the `/tmp` directory.

```
[user@host ~]$ ln newfile.txt /tmp/newfile-hlink2.txt
[user@host ~]$ ls -l newfile.txt /tmp/newfile-hlink2.txt
-rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
-rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

To determine whether two files are hard linked, use the `ls` command `-i` option to list each file's *inode number*. If the files are on the same file system and their inode numbers are the same, then the files are hard links that point to the same data file content.

```
[user@host ~]$ ls -il newfile.txt /tmp/newfile-hlink2.txt
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 newfile.txt
8924107 -rw-rw-r--. 2 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
```

Important

Hard links that reference the same file share the inode structure with the link count, access permissions, user and group ownership, time stamps, and file content. When that information is changed for one hard link, then the other hard links for the same file also show the new information. This behavior is because each hard link points to the same data on the storage device.

Even if the original file is deleted, you can still access the contents of the file provided that at least one other hard link exists. Data is deleted from storage only when the last hard link is deleted, which makes the file contents unreferenced by any hard link.

```
[user@host ~]$ rm -f newfile.txt
[user@host ~]$ ls -l /tmp/newfile-hlink2.txt
-rw-rw-r--. 1 user user 12 Mar 11 19:19 /tmp/newfile-hlink2.txt
[user@host ~]$ cat /tmp/newfile-hlink2.txt
Hello World
```

Limitations of Hard Links

Hard links have some limitations. First, you can use hard links only with regular files. You cannot use the `ln` command to create a hard link to a directory or special file.

Second, you can use hard links only if both files are on the same *file system*. The file-system hierarchy can be composed of multiple storage devices. Depending on the configuration of your system, when you change into a new directory, that directory and its contents might be stored on a different file system.

You can use the `df` command to list the directories that are on different file systems. For example, you might see the following output:

```
[user@host ~]$ df
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
devtmpfs	886788	0	886788	0%	/dev
tmpfs	902108	0	902108	0%	/dev/shm
tmpfs	902108	8696	893412	1%	/run
tmpfs	902108	0	902108	0%	/sys/fs/cgroup
/dev/mapper/rhel_rhel9--root	10258432	1630460	8627972	16%	/
/dev/sda1	1038336	167128	871208	17%	/boot
tmpfs	180420	0	180420	0%	/run/user/1000

Files in two different "Mounted on" directories and their subdirectories are on different file systems. So, in the system in this example, you can create a hard link between the `/var/tmp/link1` and `/home/user/file` files, because they are both subdirectories of the `/` directory but not of any other directory on the list. However, you cannot create a hard link between the `/boot/test/badlink` and `/home/user/file` files. The first file is in a subdirectory of the `/boot` directory (on the "Mounted on" list) and it is in the `/dev/sda1` file system. The second file is in the `/dev/mapper/rhel_rhel9--root` file system.

Create Symbolic Links

The `ln` command `-s` option creates a symbolic link, which is also called a "soft link". A symbolic link is not a regular file, but a special type of file that points to an existing file or directory.

Symbolic links have some advantages over hard links:

- Symbolic links can link two files on different file systems.
- Symbolic links can point to a directory or special file, not just to a regular file.

In the following example, the `ln -s` command creates a symbolic link for the `/home/user/newfile-link2.txt` file. The name for the symbolic link is `/tmp/newfile-symlink.txt`.

```
[user@host ~]$ ln -s /home/user/newfile-link2.txt /tmp/newfile-symlink.txt
[user@host ~]$ ls -l newfile-link2.txt /tmp/newfile-symlink.txt
-rw-rw-r--. 1 user user 12 Mar 11 19:19 newfile-link2.txt
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/newfile-link2.txt
[user@host ~]$ cat /tmp/newfile-symlink.txt
Symbolic Hello World
```

In the preceding example, the first character of the long listing for the `/tmp/newfile-symlink.txt` file is `l` (letter l) instead of `-`. This character indicates that the file is a symbolic link and not a regular file.

When the original regular file is deleted, the symbolic link still points to the file but the target is gone. A symbolic link that points to a missing file is called a "dangling symbolic link".

```
[user@host ~]$ rm -f newfile-link2.txt
[user@host ~]$ ls -l /tmp/newfile-symlink.txt
lrwxrwxrwx. 1 user user 11 Mar 11 20:59 /tmp/newfile-symlink.txt -> /home/user/newfile-link2.txt
[user@host ~]$ cat /tmp/newfile-symlink.txt
cat: /tmp/newfile-symlink.txt: No such file or directory
```

Important

One side-effect of the dangling symbolic link in the preceding example is that if you create a file with the same name as the deleted file (`/home/user/newfile-link2.txt`), then the symbolic link is no longer "dangling" and points to the new file. Hard links do not work in this way. If you delete a hard link and then use normal tools (rather than `ln`) to create a file with the same name, then the new file is not linked to the old file. Consider the following way to compare hard links and symbolic links, to understand how they work:

- A hard link points a name to data on a storage device.
- A symbolic link points a name to another name, which points to data on a storage device.

A symbolic link can point to a directory. The symbolic link then acts like the directory. If you use `cd` to change to the symbolic link, then the current working directory becomes the linked directory. Some tools might track that you followed a symbolic link to get there. For example, by default, `cd` updates your current working directory by using the name of the symbolic link rather than the name of the actual directory. If you want to update the current working directory by using the name of the actual directory, then you can use the `-P` option.

The following example creates a symbolic link called `/home/user/configfiles` that points to the `/etc` directory.

```
[user@host ~]$ ln -s /etc /home/user/configfiles
[user@host ~]$ cd /home/user/configfiles
[user@host configfiles]$ pwd
/home/user/configfiles
[user@host configfiles]$ cd -P /home/user/configfiles
[user@host etc]$ pwd
/etc
```

Guided Exercise: Make Links Between Files

In this exercise, you create hard links and symbolic links and compare the results.

Outcomes

- Create hard links and symbolic links between files.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command prepares your environment and ensures that all required resources are available.

```
[student@workstation ~]$ lab start files-make
```

Instructions

1. Use the `ssh` command to log in to the `servera` machine as the `student` user. The system's configuration supports the use SSH keys for authentication; therefore, you do not require a password.

```
2. [student@workstation ~]$ ssh student@servera
```

```
3. ...output omitted...
```

```
[student@servera ~]$
```

4. Create a hard link called `/home/student/links/file.hardlink` for the file `/home/student/files/target.file`. Verify the link count for the original file and the new linked file.

1. View the link count for the file `/home/student/files/target.file`.

```
2. [student@servera ~]$ ls -l files/target.file
```

```
3. total 4
```

```
-rw-r--r--. 1 student student 11 Mar  3 06:51 files/target.file
```

4. Create a hard link called `/home/student/links/file.hardlink`. Link it to the file `/home/student/files/target.file`.

```
5. [student@servera ~]$ ln /home/student/files/target.file \
```

```
/home/student/links/file.hardlink
```

6. Verify the link count for the original file `/home/student/files/target.file` and the new linked file, `/home/student/files/file.hardlink`. The link count should be 2 for both files.

```
7. [student@servera ~]$ ls -l files/target.file links/file.hardlink
```

```
8. -rw-r--r--. 2 student student 11 Mar  3 06:51 files/target.file
```

```
-rw-r--r--. 2 student student 11 Mar  3 06:51 links/file.hardlink
```

5. Create a symbolic link called `/home/student/tempdir` that points to the `/tmp` directory on the `servera` machine. Verify the newly created symbolic link.

1. Create a symbolic link called `/home/student/tempdir` and link it to the `/tmp` directory.

```
[student@servera ~]$ ln -s /tmp /home/student/tempdir
```

2. Use the `ls -l` command to verify the newly created symbolic link.

```
3. [student@servera ~]$ ls -l /home/student/tempdir
```

```
lrwxrwxrwx. 1 student student 4 Mar  3 06:55 /home/student/tempdir -> /tmp
```

6. Return to the workstation system as the student user.

```
7. [student@servera ~]$ exit
```

8. logout

9. Connection to servera closed.

```
[student@workstation ~]$
```

Finish

On the workstation machine, change to the student user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish files-make
```

This concludes the section.

Lab: Manage Files from the Command Line

In this lab, you efficiently create, move, and remove files and directories by using the shell and various file name matching techniques.

Outcomes

- Use wildcards to locate and manipulate files.

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command prepares your environment and ensures that all required resources are available.

```
[student@workstation ~]$ lab start files-review
```

Instructions

1. Use the `ssh` command to log in to the `serverb` machine as the `student` user. The system's configuration supports the use of SSH keys for authentication.

```
2. [student@workstation ~]$ ssh student@serverb
```

...output omitted...

Hide Solution

3. Create a directory called `project_plans` in the `Documents` directory. The `Documents` directory is placed in the `student` user's home directory. Create two empty files in the `project_plans` directory called `season1_project_plan.odf` and `season2_project_plan.odf`.

Hint: If the `~/Documents` directory does not exist, then use the `mkdir` command `-p` option to create it.

```
[student@serverb ~]$ mkdir -p ~/Documents/project_plans
[student@serverb ~]$ touch \
~/Documents/project_plans/{season1,season2}_project_plan.odf
[student@serverb ~]$ ls -lR Documents/
Documents/:
total 0
drwxr-xr-x. 2 student student 70 Mar  7 03:50 project_plans

Documents/project_plans:
total 0
-rw-r--r--. 1 student student 0 Mar  7 03:50 season1_project_plan.odf
-rw-r--r--. 1 student student 0 Mar  7 03:50 season2_project_plan.odf
```

Hide Solution

4. Create sets of empty practice files to use in this lab. If you do not immediately recognize the intended shell expansion shortcut, then use the solution to learn and practice. Use shell tab completion to locate file path

names. Create 12 files with `tv_seasonX_episodeY.ogg` names in the `/home/student` directory. Replace `X` with the season number and `Y` with that season's episode, for two seasons of six episodes each.

```
5. [student@serverb ~]$ touch tv_season{1..2}_episode{1..6}.ogg
6. [student@serverb ~]$ ls tv*
7. tv_season1_episode1.ogg  tv_season1_episode5.ogg  tv_season2_episode3.ogg
8. tv_season1_episode2.ogg  tv_season1_episode6.ogg  tv_season2_episode4.ogg
9. tv_season1_episode3.ogg  tv_season2_episode1.ogg  tv_season2_episode5.ogg
```

```
tv_season1_episode4.ogg  tv_season2_episode2.ogg  tv_season2_episode6.ogg
```

Hide Solution

10. As the author of a successful series of mystery novels, you are editing your next bestseller's chapters for publishing. Create eight files with `mystery_chapterX.odf` names. Replace `X` with the numbers 1 through 8.

```
11. [student@serverb ~]$ touch mystery_chapter{1..8}.odf
12. [student@serverb ~]$ ls mys*
13. mystery_chapter1.odf  mystery_chapter4.odf  mystery_chapter7.odf
14. mystery_chapter2.odf  mystery_chapter5.odf  mystery_chapter8.odf
```

```
mystery_chapter3.odf  mystery_chapter6.odf
```

Hide Solution

15. Use a single command to create two subdirectories called `season1` and `season2` under the `videos` directory to organize the TV episodes. Move the appropriate TV episodes into the season subdirectories. Use only two commands, and specify destinations with relative syntax.

1. Create two subdirectories called `season1` and `season2` under the `videos` directory by using a single command.

```
2. [student@serverb ~]$ mkdir -p Videos/season{1..2}
3. [student@serverb ~]$ ls Videos
```

```
season1  season2
```

4. Move the appropriate TV episodes into the season subdirectories by using only two commands.

```

5. [student@serverb ~]$ mv tv_season1* Videos/season1
6. [student@serverb ~]$ mv tv_season2* Videos/season2
7. [student@serverb ~]$ ls -R Videos
8. Videos:
9. season1  season2
10.
11. Videos/season1:
12. tv_season1_episode1.ogg  tv_season1_episode3.ogg  tv_season1_episode5.ogg
13. tv_season1_episode2.ogg  tv_season1_episode4.ogg  tv_season1_episode6.ogg
14.
15. Videos/season2:
16. tv_season2_episode1.ogg  tv_season2_episode3.ogg  tv_season2_episode5.ogg

```

```

tv_season2_episode2.ogg  tv_season2_episode4.ogg  tv_season2_episode6.ogg

```

16. Hide Solution

17. Create a two-level directory hierarchy with a single command to organize the mystery book chapters. Create the `my_bestseller` subdirectory under the Documents directory, and create the `chapters` subdirectory under the new `my_bestseller` directory. Create three more subdirectories directly under the `my_bestseller` directory with a single command. Name these subdirectories `editor`, `changes`, and `vacation`. You do not need to use the `mkdir -p` command to create parents because the `my_bestseller` parent directory exists.

1. Create the `my_bestseller` directory under the Documents directory.
Create the `chapters` directory under the `my_bestseller` directory.

```

2. [student@serverb ~]$ mkdir -p Documents/my_bestseller/chapters
3. [student@serverb ~]$ ls -R Documents
4. Documents:
5. my_bestseller  project_plans
6.
7. Documents/my_bestseller:

```



```
8. chapters
9.
10. Documents/my_bestseller/chapters:
11.
12. Documents/project_plans:
```

```
season1_project_plan.odf  season2_project_plan.odf
```

13. Create three directories called editor, changes, and vacation, under the my_bestseller directory by using a single command.

```
14. [student@serverb ~]$ mkdir Documents/my_bestseller/{editor,changes,vacation}
15. [student@serverb ~]$ ls -R Documents
16. Documents:
17. my_bestseller  project_plans
18.
19. Documents/my_bestseller:
20. changes  chapters  editor  vacation
21.
22. Documents/my_bestseller/changes:
23.
24. Documents/my_bestseller/chapters:
25.
26. Documents/my_bestseller/editor:
27.
28. Documents/my_bestseller/vacation:
29.
30. Documents/project_plans:
```

```
season1_project_plan.odf  season2_project_plan.odf
```

18. Hide Solution

19. Change to the chapters directory. Use the tilde (~) home directory shortcut to move all book chapters to the chapters directory, which is now your current directory. Use the simplest syntax to specify the destination directory.

You want to send the first two chapters to the editor for review. Move only those two chapters to the `editor` directory to avoid modifying them during the review. Starting from the `chapters` subdirectory, use brace expansion with a range to specify the chapter file names to move and a relative path for the destination directory.

While on vacation, you intend to write chapters 7 and 8. Use a single command to move the files from the `chapters` directory to the `vacation` directory. Specify the chapter file names by using brace expansion with a list of strings and without using wildcard characters.

1. Change to the `chapters` directory and use the tilde (`~`) home directory shortcut to move all book chapters to the `chapters` directory.

```
2. [student@serverb ~]$ cd Documents/my_bestseller/chapters
3. [student@serverb chapters]$ mv ~/mystery_chapter* .
4. [student@serverb chapters]$ ls
5. mystery_chapter1.odf  mystery_chapter4.odf  mystery_chapter7.odf
6. mystery_chapter2.odf  mystery_chapter5.odf  mystery_chapter8.odf
```

```
mystery_chapter3.odf  mystery_chapter6.odf
```

7. Move the first two chapters to the `editor` directory. Use brace expansion with a range to specify the chapter file names to move and a relative path for the destination directory.

```
8. [student@serverb chapters]$ mv mystery_chapter{1..2}.odf ../editor
9. [student@serverb chapters]$ ls
10. mystery_chapter3.odf  mystery_chapter5.odf  mystery_chapter7.odf
11. mystery_chapter4.odf  mystery_chapter6.odf  mystery_chapter8.odf
12. [student@serverb chapters]$ ls ../editor
```

```
mystery_chapter1.odf  mystery_chapter2.odf
```

13. Use a single command to move the chapters 7 and 8 from the `chapters` directory to the `vacation` directory. Specify the chapter file names by using brace expansion with a list of strings and without using wildcard characters.

```
14. [student@serverb chapters]$ mv mystery_chapter{7,8}.odf ../vacation
```

```
15. [student@serverb chapters]$ ls
16. mystery_chapter3.odf  mystery_chapter5.odf
17. mystery_chapter4.odf  mystery_chapter6.odf
18. [student@serverb chapters]$ ls ../vacation
```

```
mystery_chapter7.odf  mystery_chapter8.odf
```

Hide Solution

20. Change your working directory to `~/Videos/season2`, and then copy the first episode of the season to the vacation directory. Use a single `cd` command to change from your working directory to the `~/Documents/my_bestseller/vacation` directory. List its files. Use the *previous working directory* argument to return to the `season2` directory. (This argument succeeds if the last directory change with the `cd` command used only one command rather than several `cd` commands.) From the `season2` directory, copy the episode 2 file into the vacation directory. Use the shortcut again to return to the vacation directory.

1. Change your working directory to `~/Videos/season2`, and then copy the first episode of the season to the vacation directory.

```
2. [student@serverb chapters]$ cd ~/Videos/season2
```

```
[student@serverb season2]$ cp *episode1.ogg ~/Documents/my_bestseller/va
cation
```

3. Use a single `cd` command to change from your working directory to the `~/Documents/my_bestseller/vacation` directory, list its files, and use the `-` argument to return to the previous directory. Copy the episode 2 file into the vacation directory. Use the `cd` command with the `-` argument to return to the vacation directory.

```
4. [student@serverb season2]$ cd ~/Documents/my_bestseller/vacation
5. [student@serverb vacation]$ ls
6. mystery_chapter7.odf  mystery_chapter8.odf  tv_season2_episode1.ogg
7. [student@serverb vacation]$ cd -
8. /home/student/Videos/season2
9. [student@serverb season2]$ cp *episode2.ogg ~/Documents/my_bestseller/va
cation
10. [student@serverb season2]$ cd -
```

```
11. /home/student/Documents/my_bestseller/vacation
12. [student@serverb vacation]$ ls
13. mystery_chapter7.odf  tv_season2_episode1.ogg
```

```
mystery_chapter8.odf  tv_season2_episode2.ogg
```

21. Hide Solution

22. The authors of chapters 5 and 6 want to experiment with possible changes. Copy both files from the `~/Documents/my_bestseller/chapters` directory to the `~/Documents/my_bestseller/changes` directory to prevent these changes from modifying original files. Navigate to the `~/Documents/my_bestseller` directory. Use square-bracket pattern matching to specify which chapter numbers to match in the `filename` argument of the `cp` command.

```
23. [student@serverb vacation]$ cd ~/Documents/my_bestseller
24. [student@serverb my_bestseller]$ cp chapters/mystery_chapter[56].odf changes
25. [student@serverb my_bestseller]$ ls chapters
26. mystery_chapter3.odf  mystery_chapter5.odf
27. mystery_chapter4.odf  mystery_chapter6.odf
28. [student@serverb my_bestseller]$ ls changes
```

```
mystery_chapter5.odf  mystery_chapter6.odf
```

Hide Solution

29. Change your current directory to the `changes` directory and use the `date +%F` command with command substitution to copy the `mystery_chapter5.odf` file to a new file that includes the full date. Use the `mystery_chapter5_YYYY-MM-DD.odf` name format.

By using command substitution with the `date +%s` command, make another copy of `mystery_chapter5.odf`, and append the current time stamp (as the number of seconds since the epoch, 1970-01-01 00:00 UTC) to ensure a unique file name.

```
[student@serverb my_bestseller]$ cd changes
[student@serverb changes]$ cp mystery_chapter5.odf \
mystery_chapter5_$(date +%F).odf
```

```
[student@serverb changes]$ cp mystery_chapter5.odf \
mystery_chapter5_$(date +%s).odf
[student@serverb changes]$ ls
mystery_chapter5_1646644424.odf  mystery_chapter5.odf
mystery_chapter5_2022-03-07.odf  mystery_chapter6.odf
```

Hide Solution

30. After further review, you decide that you do not need the plot changes. Delete the changes directory.

If it is necessary, then navigate to the changes directory and delete all the files within the directory. You cannot delete a directory when it is the current working directory.

Change to the parent directory of the changes directory. Try to delete the empty directory by using the `rm` command without the `-r` recursive option. This attempt should fail. Finally, use the `rmdir` command to delete the empty directory, which succeeds.

When the vacation is over, you no longer need the vacation directory. Delete it by using the `rm` command with the *recursive* option.

When finished, return to the student user's home directory.

1. Delete the changes directory. Change to the parent directory of the changes directory, and try to delete the empty directory by using the `rm` command without the `-r` recursive option, which should fail. Use the `rmdir` command to delete the empty directory.

```
2. [student@serverb changes]$ rm mystery*
3. [student@serverb changes]$ cd ..
4. [student@serverb my_bestseller]$ rm changes
5. rm: cannot remove 'changes': Is a directory
6. [student@serverb my_bestseller]$ rmdir changes
7. [student@serverb my_bestseller]$ ls
```

```
chapters  editor  vacation
```

8. Delete the vacation directory by using the `rm` command with the `-r` option. Return to the student user's home directory.

```
9. [student@serverb my_bestseller]$ rm -r vacation
10. [student@serverb my_bestseller]$ ls
11. chapters  editor
12. [student@serverb my_bestseller]$ cd
```

```
[student@serverb ~]$
```

Hide Solution

31. Create a hard link to

the `~/Documents/project_plans/season2_project_plan.odf` file called `~/Documents/backups/season2_project_plan.odf.back`. A hard link protects against accidental deletion of the original file and keeps the backup file updated as you change the original file.

Hint: If the `~/Documents/backups` directory does not exist, then use the `mkdir` command to create it.

1. Create a hard link to the `~/Documents/project_plans/season2_project_plan.odf` file called `~/Documents/backups/season2_project_plan.odf.back`.

```
2. [student@serverb ~]$ mkdir ~/Documents/backups
3. [student@serverb ~]$ ln ~/Documents/project_plans/season2_project_plan.o
df \
4. ~/Documents/backups/season2_project_plan.odf.back
5. [student@serverb ~]$ ls -lR ~/Documents/
6. /home/student/Documents/:
7. total 0
8. drwxr-xr-x. 2 student student 43 Mar  7 04:18 backups
9. drwxr-xr-x. 4 student student 36 Mar  7 04:16 my_bestseller
10. drwxr-xr-x. 2 student student 70 Mar  7 03:50 project_plans
11.
12. /home/student/Documents/backups:
13. total 0
14. -rw-r--r--. 2 student student 0 Mar  7 03:50 season2_project_plan.odf.ba
ck
```

```
15.
16. /home/student/Documents/my_bestseller:
17. total 0
18. drwxr-xr-x. 2 student student 118 Mar  7 04:07 chapters
19. drwxr-xr-x. 2 student student  62 Mar  7 04:06 editor
20.
21. /home/student/Documents/my_bestseller/chapters:
22. total 0
23. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter3.odf
24. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter4.odf
25. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter5.odf
26. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter6.odf
27.
28. /home/student/Documents/my_bestseller/editor:
29. total 0
30. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter1.odf
31. -rw-r--r--. 1 student student 0 Mar  7 03:56 mystery_chapter2.odf
32.
33. /home/student/Documents/project_plans:
34. total 0
35. -rw-r--r--. 1 student student 0 Mar  7 03:50 season1_project_plan.odf
```

```
-rw-r--r--. 2 student student 0 Mar  7 03:50 season2_project_plan.odf
```

Notice that the link count is 2 for
both season2_project_plan.odf.back and season2_project_plan.odf files.

36. Return to the workstation system as the student user.

```
37. [student@serverb ~]$ exit
38. logout
39. Connection to serverb closed.
```

```
[student@workstation ~]$
```

Hide Solution

Evaluation

As the `student` user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade files-review
```

Finish

On the workstation machine, change to the `student` user home directory and use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish files-review
```

This concludes the section.